CS4234 Mini-Project

Approximation Algorithms for Bin Packing

National University of Singapore
School of Computing

Oh Chundai, Daron
Wang Xiyu
Xu Zhehao
Chen Yixun

November 2026

1 Introduction

The **Bin Packing Problem** is a classical combinatorial optimization problem with applications in resource allocation, scheduling, and storage management. Given a collection of items with sizes $a_1, \ldots, a_n \in (0, 1]$, the objective is to pack them into the minimum number of unit-capacity bins. Since finding an optimal packing is NP-hard, approximation algorithms are studied to obtain near-optimal solutions efficiently.

Fernández de la Vega and Lueker (1981) [1] developed an **asymptotic polynomial-time approximation scheme (APTAS)** for bin packing, achieving an approximation guarantee of $(1+\varepsilon)\,\mathrm{OPT}(I) + O(\varepsilon^{-2})$ for any fixed $\varepsilon > 0$. Subsequently, Karmarkar and Karp (1982) [3] introduced an improved algorithm based on linear programming, achieving an additive approximation guarantee of $\mathrm{OPT}(I) + O(\log^2 \mathrm{OPT}(I))$.

We first review greedy packing algorithms, configuration formulations, LP rounding, and a partial order on bin packing instances. These tools support our analysis of the de la Vega–Lueker and Karmarkar–Karp algorithms, focusing on their approximation guarantees and running times.

2 Preliminaries

2.1 Notation

A bin packing instance is represented by a multiset $I = \{a_1, \ldots, a_n\}$ of n items, where $a_i \in (0, 1]$ denotes the size of item i. Alternatively, if the instance contains m distinct item sizes $y_1, \ldots, y_m$, with n_j items of size y_j, we may write I in the following representation: $I = \{y_1 \times n_1, y_2 \times n_2, \ldots, y_m \times n_m\}$, $\sum_{j=1}^{m} n_j = n$. Here, $y_j \times n_j$ denotes n_j copies of the item size y_j, rather than exponentiation. We denote the total size of the instance by $\mathrm{SIZE}(I) = \sum_{i=1}^{n} a_i = \sum_{j=1}^{m} n_j y_j$. Furthermore, $\mathrm{OPT}(I)$ denotes the minimum number of bins required to pack I, while $\mathrm{ALG}(I)$ denotes the number of bins used by an algorithm ALG on instance I. We use $\varepsilon > 0$ as an approximation parameter.

2.2 Greedy Algorithms

2.2.1 Definitions

The greedy algorithms considered here process items in the given order, placing each item according to a local rule without moving previously packed items. They differ in which existing bins they consider. Both de la Vega–Lueker and Karmarkar–Karp use greedy packing in some steps of their algorithms.

Next-Fit (NF). Next-Fit keeps only the most recently opened bin available. If the next item does not fit, it closes that bin permanently and opens a new one. See Algorithm 2 in Appendix E for the pseudocode.

First-Fit (FF). First-Fit places each item in the first bin, in opening order, with enough remaining capacity, opening a new bin only if none fits. See Algorithm 3 in Appendix F for the pseudocode.

Best-Fit (BF). Best-Fit places each item in the fullest bin with enough remaining capacity, opening a new bin only if none fits; ties are broken by opening order.

2.2.2 Approximation Guarantees

Next-Fit's bound in terms of total item size will be used in the later LP rounding and Karmarkar–Karp analyses.

Pairing consecutive bins. Let $b = \mathrm{NF}(I)$ for a nonempty instance I, and let ℓ_j be the load of bin B_j. If x_{j+1} is the size of the item that opens B_{j+1}, then it did not fit in B_j, so

$$\ell_j + \ell_{j+1} \geq \ell_j + x_{j+1} > 1 \qquad (1 \leq j < b).$$

Thus, consecutive bins have combined load greater than 1. Pair the bins as $(B_1, B_2), (B_3, B_4), \ldots$, leaving the last bin unpaired if b is odd. There are $\lfloor b/2 \rfloor$ pairs, so

$$\mathrm{OPT}(I) \geq \mathrm{SIZE}(I) \geq \sum_{j=1}^{\lfloor b/2 \rfloor} (\ell_{2j-1} + \ell_{2j}) > \left\lfloor \frac{b}{2} \right\rfloor \qquad (b \geq 2).$$

For $b = 1$, we also have $\mathrm{SIZE}(I) > 0 = \lfloor b/2 \rfloor$. Since $\lfloor b/2 \rfloor$ and $\mathrm{OPT}(I)$ are integers, $\lfloor b/2 \rfloor \leq \mathrm{OPT}(I) - 1$, giving $\mathrm{NF}(I) \leq 2\,\mathrm{OPT}(I) - 1$.

The same pairing also gives a bound in terms of total item size: using $\lfloor b/2 \rfloor \geq (b-1)/2$ and rearranging yields

$$\mathrm{NF}(I) \leq 2\,\mathrm{SIZE}(I) + 1 \leq 2\,\mathrm{OPT}(I) + 1, \tag{1}$$

which also holds for the empty instance, where $\mathrm{NF}(I) = 0$.

Table 1 compares the Next-Fit bound proved above with the guarantees for FF and BF. The absolute ratio applies to all nonempty instances, while the asymptotic ratio describes the worst case as $\mathrm{OPT}(I)$ grows.

Algorithm	Bound for nonempty I	Absolute ratio	Asymptotic ratio	Reference
NF	$2\,\mathrm{OPT}(I) - 1$	2	2	Proof above
FF	$\lfloor 1.7\,\mathrm{OPT}(I) \rfloor$	1.7	1.7	[5]
BF	$\lfloor 1.7\,\mathrm{OPT}(I) \rfloor$	1.7	1.7	[5]

Table 1: Worst-case guarantees for the three greedy algorithms. Each bound is an upper bound on the number of bins used.

2.3 Configuration Formulations and LP Relaxation

The configuration-based formulation provides a common starting point for the approximation techniques developed by both de la Vega–Lueker and Karmarkar–Karp.

A **bin configuration** specifies the number of slots of each item size in a single unit-capacity bin. For distinct item sizes $y_1, \ldots, y_m$, a configuration is a nonzero vector $c = (c_1, \ldots, c_m) \in \mathbb{Z}^m_{\geq 0}$ satisfying

$$\sum_{j=1}^{m} c_j y_j \leq 1.$$

Here, c_j denotes the number of slots for items of size y_j. Two configurations are identical if their slot counts agree for every size.

The count c_j need not be bounded by the number n_j of items of that size in the instance. Configurations describe possible bin layouts; the covering constraints ensure that the selected layouts provide enough slots for the actual items. Excess slots may remain unused. The number of feasible configurations can be very large, and enumerating all of them need not take polynomial time in the input size.

2.3.1 Integer Linear Programming Formulation

Consider a Bin Packing instance I containing n items of m distinct sizes $y_1, \ldots, y_m$, with n_j items of size y_j. Let $\mathcal{C} = \{C_1, \ldots, C_q\}$ denote the set of all feasible non-empty bin configurations. For each configuration C_i, let c_{ij} denote the number of items of size y_j contained in that configuration. By feasibility, $\sum_{j=1}^{m} c_{ij} y_j \leq 1$, $\forall i \in \{1, \ldots, q\}$. We represent a packing by a vector $\mathbf{a} = (a_1, \ldots, a_q)$, where a_i denotes the number of bins assigned to configuration C_i. The Bin Packing problem can then be formulated as the following Integer Linear Program (ILP):

$$\begin{aligned} \min_{\mathbf{a}} \quad & \sum_{i=1}^{q} a_i \\ \text{subject to} \quad & \sum_{i=1}^{q} a_i c_{ij} \geq n_j, \quad \forall j \in \{1, \ldots, m\}, \\ & a_i \in \mathbb{Z}_{\geq 0}, \qquad\quad \forall i \in \{1, \ldots, q\}. \end{aligned} \tag{2}$$

The objective minimizes the total number of bins used. For each item size y_j, the corresponding constraint ensures that the selected configurations provide sufficient capacity to accommodate all n_j items of that size.

Each feasible integer vector $\mathbf{a} = (a_1, \ldots, a_q)$ corresponds to a feasible packing of I, where a_i bins are assigned to configuration C_i. Any excess item slots can be left unused.

For a feasible solution produced by an algorithm, we denote the number of bins used as $\mathrm{ALG}(I) = \sum_{i=1}^{q} a_i$. Thus, minimizing the ILP objective is equivalent to minimizing the number of bins required to pack I, and its optimal value is $\mathrm{OPT}(I)$.

However, as discussed previously, the number of feasible configurations q can be exponential in n. Consequently, even constructing the complete ILP may require exponentially many variables. This motivates considering a restricted version of Bin Packing in which the number of feasible configurations can be bounded more tightly.

2.3.2 Restricted Bin Packing: Bounded Configurations

Consider a restricted Bin Packing instance in which every item has size at least $\varepsilon > 0$. Since each bin has unit capacity, it can contain at most $k = \lfloor 1/\varepsilon \rfloor$ items. Under this restriction, the number of feasible configurations can be bounded in terms of k and the number of distinct item sizes m.

By Lemma A.1, proved in Appendix A, the number of non-empty feasible bin configurations satisfies

$$q \leq \binom{k+m}{m} - 1.$$

Consequently, for fixed k and m, the configuration ILP in Section 2.3.1 contains a bounded number of variables, independent of the total number of items n.

2.3.3 Linear Programming Relaxation

Although restricting the number of configurations reduces the size of the ILP formulation, solving the resulting integer program optimally remains computationally challenging. Hence, a natural approach is to relax the integrality constraints $a_i \in \mathbb{Z}_{\geq 0}$ to $a_i \geq 0$, obtaining a Linear Programming (LP) relaxation.

Both de la Vega–Lueker and Karmarkar–Karp exploit configuration-based formulations and LP relaxations, but they employ rounding techniques to construct feasible integral packings from the relaxed LP solution. In the following sections, we examine their respective techniques and analyze their approximation guarantees and running-time complexities.

Let $\mathbf{a}^* = (a_1^*, \ldots, a_q^*)$ be an optimal basic feasible solution to the LP relaxation, with objective value $\mathrm{LIN}(I) = \sum_{i=1}^{q} a_i^*$. The following lemma guarantees that such a solution exists and bounds the number of configurations it uses.

Lemma 2.1 (Nonzero Variables in the LP Solution)**.** *The LP relaxation of Equation (2) has an optimal basic feasible solution (equivalently, an optimal vertex solution) with at most m nonzero variables, where m is the number of distinct item sizes.*

The detailed proof is given in Appendix D.

De La Vega–Lueker's Approach By Lemma 2.1, rounding up each variable of $\mathbf{a}^*$ produces an integral solution while introducing at most m additional bins.

Rounding up the LP solution. We construct an integer vector $\hat{\mathbf{a}} = (\hat{a}_1, \ldots, \hat{a}_q)$ by setting $\hat{a}_i = \lceil a_i^* \rceil$, $\forall i \in \{1, \ldots, q\}$. Since $\hat{a}_i \geq a_i^*$ for every i, the rounded solution continues to satisfy all covering constraints: $\sum_{i=1}^{q} \hat{a}_i c_{ij} \geq \sum_{i=1}^{q} a_i^* c_{ij} \geq n_j$, $\forall j \in \{1, \ldots, m\}$.

Hence, $\hat{\mathbf{a}}$ is a feasible integer solution to the ILP. Moreover, each non-zero variable increases by less than one during rounding. Since there are at most m such variables, the total number of bins increases by at most m.

Greedy Deflation. Although the rounded solution satisfies the covering constraints, it may contain more items of a particular size than required. We therefore perform a polynomial-time **deflation** procedure that removes excess items by replacing selected configurations with smaller feasible configurations. A detailed batch-deflation algorithm, together with an illustrative example and correctness argument, is provided in Appendix B.

More generally, rounding an arbitrary optimal LP solution introduces at most q additional bins, giving $\mathrm{ALG}(I) \leq \mathrm{LIN}(I) + q \leq \mathrm{OPT}(I) + q$ after deflation. Choosing an optimal vertex solution as above strengthens the additive bound to m, because at most m variables are nonzero.

Karmarkar–Karp's Approach Karmarkar and Karp split an optimal basic feasible solution $\mathbf{a}^*$ into its integral and fractional parts, $\mathbf{a}^* = \mathbf{p}^* + \mathbf{f}^*$, where $p_i^* = \lfloor a_i^* \rfloor$ and $f_i^* = a_i^* - p_i^* \in [0, 1)$ for $i = 1, \ldots, q$. The integral part provides a partial packing using $\sum_{i=1}^{q} p_i^*$ bins, with excess item slots left unused. The remaining items form an instance I' with $n'_j = \max\left\{0,\, n_j - \sum_{i=1}^{q} p_i^* c_{ij}\right\} \leq \sum_{i=1}^{q} f_i^* c_{ij}$, $j = 1, \ldots, m$. Thus, $\mathbf{f}^*$ is a feasible solution for LP relaxation of I'. It is also optimal: a cheaper fractional packing of I', combined with $\mathbf{p}^*$, would contradict the optimality of $\mathbf{a}^*$. Hence, $\mathrm{LIN}(I') = \sum_{i=1}^{q} f_i^*$.

We construct two packings of I'. By Lemma 2.1, $\mathbf{a}^*$ has at most m nonzero components. Opening one bin of configuration i for each $f_i^* > 0$ and deleting excess slots therefore gives a packing with at most m bins. Alternatively, the greedy bound gives a packing with at most $2\,\mathrm{SIZE}(I') + 1$ bins. Both constructions take polynomial time; choosing the smaller packing yields $\mathrm{ALG}(I') \leq \min\{m,\, 2\,\mathrm{SIZE}(I') + 1\} \leq \mathrm{SIZE}(I') + \dfrac{m+1}{2}$.

For any instance I, bin capacity and the covering constraints imply $\mathrm{LIN}(I) = \sum_{i=1}^{q} a_i^* \geq \sum_{i=1}^{q} a_i^* \sum_{j=1}^{m} y_j c_{ij} = \sum_{j=1}^{m} y_j \sum_{i=1}^{q} a_i^* c_{ij} \geq \sum_{j=1}^{m} y_j n_j = \mathrm{SIZE}(I)$. Combining the principal and residual packings gives $\mathrm{OPT}(I) \leq \mathrm{ALG}(I) = \sum_{i=1}^{q} p_i^* + \mathrm{ALG}(I') = \mathrm{LIN}(I) - \mathrm{LIN}(I') + \mathrm{ALG}(I') \leq \mathrm{LIN}(I) - \mathrm{LIN}(I') + \mathrm{SIZE}(I') + \dfrac{m+1}{2} \leq \mathrm{LIN}(I) + \dfrac{m+1}{2}$.

2.4 Partial Order in Bin Packing Instance

Both papers introduce a partial order $\preceq$ on bin packing instances. Given two instances $I = \{a_1, \ldots, a_n\}$ and $J = \{b_1, \ldots, b_m\}$, we write $I \preceq J$ if there exists an injective mapping $f : \{1, \ldots, n\} \to \{1, \ldots, m\}$ such that $a_i \leq b_{f(i)}$, $\forall i \in \{1, \ldots, n\}$. In other words, every item in I can be matched to a distinct item in J of at least the same size.

This definition yields Lemma C.1, which states that if $I \preceq J$, then $\mathrm{OPT}(I) \leq \mathrm{OPT}(J)$.

3 De La Vega–Lueker Algorithm

Fernández de la Vega and Lueker [1] reduce bin packing to the restricted problem of Section 2.3.2, in which every item has a minimum size and the number of distinct sizes is bounded. The reduction temporarily sets aside the small items and groups the large items by rank. We solve the configuration LP on the resulting rounded instance, recover a packing of the original large items, and finally reinsert the small items greedily. For an instance U, we write $m(U)$ for the number of distinct sizes in U.

3.1 Elimination and linear grouping

Choice of threshold. Let $\varepsilon > 0$ be the approximation parameter. We use a cutoff δ and a group count t given by

$$\delta = \frac{\varepsilon}{\varepsilon + 2}, \qquad t = \lceil \delta^{-2} \rceil.$$

The analysis loses two multiplicative factors. Packing one group of large items separately costs a factor $1 + \delta$ (Lemma 3.1), and reinserting small items only guarantees bin loads above $1 - \delta$ (Claim 3.3). The cutoff is the root of $(1 + \delta)/(1 - \delta) = 1 + \varepsilon$, so neither loss exceeds $1 + \varepsilon$. The group count is the smallest integer with $\delta t \geq 1/\delta$, which is exactly what makes one group at most a δ-fraction of the optimum.

Elimination. Put the items of size less than δ in S and the remaining items in L. Write $N = |L| = th + r$, where $h = \lfloor N/t \rfloor$ and $0 \leq r < t$.

Linear grouping. List L in non-increasing order of size. Set aside the first r items as R, and partition the remaining th items into t consecutive groups $G_1, \ldots, G_t$ of h items each. If $h \geq 1$, for each $j = 2, \ldots, t$, round every item of G_j up to $\min G_{j-1}$, the smallest size in the preceding group. Denote the resulting rounded group by G'_j, and define $G'_j = \{\min G_{j-1} \times h\}$, $J = \biguplus_{j=2}^{t} G'_j$, where $\biguplus$ denotes multiset union. The first group G_1 has no predecessor, so it is not rounded; its items are packed separately, one per bin. If $h = 0$, set $R = L$ and $J = \varnothing$.

Lemma 3.1 (Linear grouping)**.** *Suppose $h \geq 1$. Then*

$$J \preceq L, \qquad m(J) \leq t - 1, \qquad \min J \geq \delta, \qquad h \leq \delta \operatorname{OPT}(I).$$

Moreover, for $j = 2, \ldots, t$, every original item of G_j fits in the slot of its rounded copy in G'_j.

Proof. For each $j = 2, \ldots, t$, every item of G'_j has size $\min G_{j-1}$. Since L is listed in non-increasing order, every original item of G_j has size at most $\min G_{j-1}$, so it fits in the slot of its rounded copy in G'_j.

To prove $J \preceq L$, match the h items of each G'_j one-to-one with the h items of G_{j-1}. Every target item has size at least $\min G_{j-1}$, so $G'_j \preceq G_{j-1}$. The target groups $G_1, \ldots, G_{t-1}$ contain distinct item copies. Thus, combining these matchings gives an injective mapping from $J = \biguplus_{j=2}^{t} G'_j$ into L, proving $J \preceq L$.

Each rounded group G'_j contributes the single size $\min G_{j-1}$, so $m(J) \leq t - 1$. Rounding only increases sizes, so every size in J is at least δ.

Finally, every item of L has size at least δ, so $\delta N \leq \operatorname{SIZE}(L) \leq \operatorname{SIZE}(I) \leq \operatorname{OPT}(I)$. Using $t \geq \delta^{-2}$,

$$h \leq \frac{N}{t} \leq \frac{\operatorname{OPT}(I)}{\delta t} \leq \delta \operatorname{OPT}(I). \qquad \square$$

Since $J \preceq L \preceq I$, Lemma C.1 gives $\operatorname{OPT}(J) \leq \operatorname{OPT}(I)$. Moreover, J is a restricted instance with at most $t - 1$ distinct sizes, each at least δ (Section 2.3.2 with δ in place of ε).

Worked grouping. Suppose $h = 2$ and the first three groups are

$$G_1 = \{0.60, 0.54\}, \qquad G_2 = \{0.49, 0.46\}, \qquad G_3 = \{0.41, 0.39\}.$$

The rounded groups are $G_2' = \{0.54 \times 2\}$ and $G_3' = \{0.46 \times 2\}$. These groups contribute $G_2' \uplus G_3' = \{0.54 \times 2, 0.46 \times 2\}$ to J, while G_1 is packed in two separate bins. A bin with one 0.54 slot and one 0.46 slot can hold, for example, the original items 0.49 and 0.41. Figure 2 in Appendix I illustrates the rounding and the matching to predecessor slots.

3.2 Packing and reinserting small items

Packing the large items. Solve the configuration LP relaxation on J and take an optimal basic feasible solution $\mathbf{a}^*$. Choosing a vertex solution ensures that rounding adds at most $m(J)$ bins, rather than the general bound of q for an arbitrary optimal LP solution, where q is the number of configurations of J. Following the de la Vega–Lueker approach in Section 2.3.3, round up each configuration count and apply greedy deflation to remove excess slots. Fill every slot with the original item of its rounded copy. Pack each item of G_1 and of R in a separate bin, omitting G_1 when $h = 0$. Let B_0 be the number of bins used at this point.

Reinserting the small items. Use a Next-Fit-style scan of the existing bins, starting at the first bin. Process S in input order, advancing the pointer only when an item does not fit and opening a new bin only after passing the last bin. The pointer never moves backward. See Algorithm 4 in Appendix G for the detailed pseudocode.

Claim 3.2 (Packing the large items)**.** *The bin count before reinsertion satisfies*

$$B_0 \le (1+\delta)\,\mathrm{OPT}(I) + 2(t-1).$$

Proof. If $h = 0$, the LP packing is empty and all large items lie in R, so $B_0 = r \le t-1$. Assume $h \ge 1$.

LP part. By Lemma 2.1, the optimal basic feasible solution $\mathbf{a}^*$ for J has at most $m(J)$ positive entries a_j^*. Rounding up each positive entry adds less than one bin, while zero entries stay zero. Thus, $\sum_j \lceil a_j^* \rceil \le \sum_j a_j^* + m(J) = \mathrm{LIN}(J) + m(J) \le \mathrm{OPT}(J) + m(J)$. Deflation never increases the bin count. By Lemma 3.1, replacing each rounded item with its original keeps every bin feasible.

Separate part. The first group G_1 contains h items, and the remainder R contains r items. Packing each of these items in its own bin uses $h + r$ bins.

Combine. Together, these give $B_0 \le \sum_j \lceil a_j^* \rceil + h + r$. Using $\mathrm{OPT}(J) \le \mathrm{OPT}(I)$, $m(J) \le t-1$, $h \le \delta\,\mathrm{OPT}(I)$ and $r \le t-1$,

$$B_0 \le \mathrm{OPT}(J) + m(J) + h + r \le \mathrm{OPT}(I) + (t-1) + \delta\,\mathrm{OPT}(I) + (t-1). \qquad \square$$

Claim 3.3 (Reinserting the small items)**.** *If reinsertion opens a new bin, the final bin count B satisfies*

$$B \le \frac{\mathrm{SIZE}(I)}{1-\delta} + 1.$$

Proof. Consider the last bin opened during reinsertion. Since the pointer never moves backward, it had already left each of the other $B-1$ bins at that moment. The pointer leaves a bin only when some small item $a < \delta$ fails to fit in it, so that bin already had load greater than $1 - a > 1 - \delta$. Loads never decrease afterwards. Summing the loads of these $B-1$ bins gives $\mathrm{SIZE}(I) \ge (B-1)(1-\delta)$, and rearranging yields the bound. $\square$

3.3 Approximation guarantee and running time

Theorem 3.4. *For any $\varepsilon > 0$, the De La Vega–Lueker algorithm returns a feasible packing of I with*

$$\mathrm{ALG}(I) \le (1+\varepsilon)\,\mathrm{OPT}(I) + 2(t-1), \qquad t = \lceil (1 + 2/\varepsilon)^2 \rceil.$$

In particular, $\mathrm{ALG}(I) \le (1+\varepsilon)\,\mathrm{OPT}(I) + O(\varepsilon^{-2})$ *for* $0 < \varepsilon \le 1$. *Its running time is* $C_\varepsilon + O(n \log t)$ *in the arithmetic model, where* C_ε *depends only on* ε.

Proof. **Feasibility.** Every large item lies in its own bin or in the slot of its rounded copy, which it fits by Lemma 3.1. Every small item is placed only in a bin where it fits.

Approximation guarantee. Since $\delta = \varepsilon/(\varepsilon+2)$, we directly have $1+\delta \le 1+\varepsilon$ and $1/(1-\delta) = 1+\varepsilon/2 \le 1+\varepsilon$. Also, $t \ge 2$ since $\delta < 1$. If reinsertion uses only the initial bins, then $B = B_0$, and Claim 3.2 gives $B \le (1+\varepsilon)\,\mathrm{OPT}(I) + 2(t-1)$. If reinsertion opens a new bin, Claim 3.3 and $\mathrm{SIZE}(I) \le \mathrm{OPT}(I)$ give

$$B \le \frac{\mathrm{OPT}(I)}{1-\delta} + 1 \le (1+\varepsilon)\,\mathrm{OPT}(I) + 2(t-1).$$

Since $1/\delta = 1 + 2/\varepsilon$, we have $t = \lceil (1+2/\varepsilon)^2 \rceil$. For $0 < \varepsilon \le 1$, $\delta \ge \varepsilon/3$, so $t \le 9\varepsilon^{-2} + 1$ and the additive term is $O(\varepsilon^{-2})$. For fixed ε, its contribution to $\mathrm{ALG}(I)/\,\mathrm{OPT}(I)$ tends to zero as $\mathrm{OPT}(I)$ grows, giving an asymptotic approximation ratio of at most $1+\varepsilon$.

Running time. Separating the small items takes $O(n)$ time. Full sorting is unnecessary: items within each group may remain unordered. To find the boundaries at ranks $r, r+h, \ldots, r+(t-1)h$ (omitting rank zero), use Quickselect with median-of-medians pivots for worst-case linear-time selection of the middle requested boundary, partition, and recurse on each side, breaking ties by input position. Each level costs $O(n)$ over disjoint sublists, and each subproblem retains at most half the requested boundaries, giving $O(\log t)$ levels. Scanning for all group minima takes $O(n)$, so grouping takes $O(n \log t)$ time.

In J, each configuration contains at most $k = \lfloor 1/\delta \rfloor$ items, and there are at most $t-1$ distinct sizes. By Lemma A.1, the number of non-empty configurations is at most $q \le \binom{k+t-1}{t-1} - 1$. Both the number of configurations and the number of LP constraints are bounded by functions of ε alone. Enumerating configurations, solving the LP (for example, by enumerating candidate bases and solving linear systems of bounded dimension) and batch deflation therefore take at most C_ε arithmetic operations. Filling the slots takes $O(n)$ time. Reinsertion also takes $O(n)$ time, because each small item is placed once and the pointer passes each bin at most once. The total is $C_\varepsilon + O(n \log t)$, which is linear in n for fixed ε. Together with the guarantee above, this meets the definition of an APTAS: polynomial running time for each fixed $\varepsilon > 0$ and an asymptotic approximation ratio of at most $1+\varepsilon$. □

4 Karmarkar–Karp Algorithm

We study Algorithm 2 of Karmarkar and Karp [3, Theorem 4]. We use the configuration formulation in Section 2.3.1 and its LP relaxation in Section 2.3.3. For an instance U, let $m(U)$ count its distinct sizes. We write the covering constraints as $Ax \ge b$, $x \ge 0$, where $A_{ji} = c_{ij}$: row j corresponds to item size y_j and column i to configuration C_i. The variable x_i is the fractional usage of configuration C_i, denoted by a_i in the earlier formulation. The demand vector $b = (n_1, \ldots, n_{m(U)})^\top$ records the number of items of each size in U. For the partial order defined in Section 2.4, we also use fractional monotonicity: $U \preceq V$ implies $\mathrm{LIN}(U) \le \mathrm{LIN}(V)$ [3, Section 4].

Karmarkar and Karp's innovative approach combines geometric grouping with LP rounding and repeated processing of the residual instance. In each round, let R denote the original items still unpacked. Geometric grouping produces a rounded instance J for the LP and a collection E of original items excluded from J during grouping, which are packed separately. The demand vector b records the multiplicities of the rounded sizes in J.

After constructing J, we use its LP solution to pack some of the original items. The unpacked originals form the residual instance processed in the next round.

Let I contain n items with rational sizes. For $\mathrm{SIZE}(I) > 2$, set aside items of size at most $g = 1/\,\mathrm{SIZE}(I)$ until the end and put the rest in R. The cutoff bounds group lengths, and $g\,\mathrm{SIZE}(I) = 1$ limits the cost of reinserting small items. We use groups with target size $k = 2$ and stop when the residual size is at most $1 + 2L$, where $L = \lceil \log_2(2\,\mathrm{SIZE}(I)) \rceil$. This integer threshold upper-bounds the paper's $1 + 2\ln(1/g)$. We choose this threshold to simplify the algebra in the proof while preserving the asymptotic approximation guarantee.

4.1 Geometric grouping

The version of geometric grouping used in Algorithm 2 forms groups by total size. As sizes decrease, more items may be needed to reach the same target. Trimming each group to the length of its predecessor lets the rounded items fit into the predecessor's slots. Figure 3 in Appendix J illustrates the groups G_i, the items set aside in E, and the rounded instance J. The dotted connections show how items of J can be matched to slots in the preceding groups.

Lemma 4.1 (Geometric grouping)**.** *Suppose all items of U have size greater than g, where $0 < g < 1$. For an integer $k \geq 2$, the grouping below produces a rounded instance J and a multiset E of items to pack separately, satisfying*

$$J \preceq U \preceq J \uplus E, \qquad m(J) \leq \frac{\mathrm{SIZE}(U)}{k}, \qquad \mathrm{SIZE}(E) \leq 2k + 1 + (k+1)\ln(2/g).$$

Here $\uplus$ denotes multiset union.

Proof. Sort the items of U in non-increasing order of size. Starting from the largest remaining item, collect consecutive items until their total size first reaches or exceeds k; these items form G_1. Repeat this procedure on the remaining items to form $G_2, G_3, \ldots$. Stop when the total size of all remaining items is less than k. Let q be the number of complete groups formed, and let H denote the remaining items, possibly none. Thus, each complete group G_i has total size at least k, whereas $\mathrm{SIZE}(H) < k$.

If $q = 0$, then $\mathrm{SIZE}(U) < k$, so no complete group can be formed. In this case, set $E = U$ and $J = \varnothing$: all items are assigned to the subinstance packed separately, and there are no rounded items for the LP. These choices satisfy the lemma's bounds.

Otherwise, place the first complete group G_1 and the incomplete suffix H into E. The later groups will contribute items to J after trimming and rounding; their trimmed items will also be added to E.

For each complete group, let $\ell_i = |G_i|$. Since its total size first reaches k upon adding the last item, whose size is at most one, we have $k \leq \mathrm{SIZE}(G_i) < k + 1$. Moreover, items in later groups are no larger than those in earlier groups, so reaching the same target k requires at least as many items. Thus, $\ell_{i-1} \leq \ell_i$.

For each $i \geq 2$, put the $d_i = \ell_i - \ell_{i-1}$ smallest items of G_i into E. Round its remaining ℓ_{i-1} items up to $\max G_i$ and add them to J, keeping the identity of each original. Every item in G_{i-1} is at least as large as $\max G_i$, so these rounded items can be matched to G_{i-1}. The predecessor groups are disjoint, giving $J \preceq U$. Matching originals to their rounded copies or to themselves in E gives $U \preceq J \uplus E$. The q completed groups each have total size at least k, so $qk \leq \mathrm{SIZE}(U)$. Each group after G_1 contributes at most one distinct rounded size, giving $m(J) \leq q - 1 \leq \mathrm{SIZE}(U)/k$.

Let D_i denote the d_i items trimmed from G_i. The smallest d_i items account for at most d_i/ℓ_i of the total size of G_i. Since $1 - u \leq -\ln u$ for $0 < u \leq 1$, we have

$$\mathrm{SIZE}(D_i) \leq (k+1)\frac{d_i}{\ell_i} \leq (k+1)\ln\frac{\ell_i}{\ell_{i-1}}.$$

The logarithms sum to $\ln(\ell_q/\ell_1)$. Every item has size at most one, so $\ell_1 = |G_1| \geq \mathrm{SIZE}(G_1) \geq k$. Also, $\ell_q - 1 < k/g$ because the items before the last in G_q have total size below k and each exceeds g. Thus $\ell_q/\ell_1 < 1/g + 1/k < 2/g$. Adding $\mathrm{SIZE}(G_1) < k + 1$ and $\mathrm{SIZE}(H) < k$ gives the bound on $\mathrm{SIZE}(E)$. □

4.2 Packing the integer part

Karmarkar and Karp give a polynomial-time algorithm that, for an instance I and a tolerance $h > 0$, computes a basic feasible solution to the LP relaxation with cost at most $\mathrm{LIN}(I) + h$ [3, Theorem 1]. Applying it to J with $h = 1$ gives x with $\sum_j x_j \leq \mathrm{LIN}(J) + 1$. By the argument bounding the number of nonzero variables in Lemma 2.1, x has at most $m(J)$ nonzero components.

Applying the integer-part packing from Section 2.3.3, we open $\lfloor x_j \rfloor$ bins of each layout and fill their slots with available originals of the corresponding rounded type. The unpacked originals form the next residual instance R'.

Lemma 4.2 (Residual demand)**.** *Let R' consist of the original items left unpacked after this step. If $X = \sum_j \lfloor x_j \rfloor$ bins were opened, then*

$$\mathrm{SIZE}(R') \le \mathrm{LIN}(R') \le \sum_j (x_j - \lfloor x_j \rfloor) \le m(J), \qquad X + \mathrm{LIN}(R') \le \mathrm{LIN}(J) + 1.$$

Proof. The residual-covering argument in Section 2.3.3 and the bound on the number of nonzero variables from Lemma 2.1 give

$$\mathrm{SIZE}(R') \le \mathrm{LIN}(R') \le \sum_j (x_j - \lfloor x_j \rfloor) \le m(J),$$

since each unpacked original fits its corresponding rounded slot. Adding $X = \sum_j \lfloor x_j \rfloor$ yields

$$X + \mathrm{LIN}(R') \le \sum_j x_j \le \mathrm{LIN}(J) + 1. \qquad \square$$

Corollary 4.3 (One round)**.** *Starting from residual R, one round with $k = 2$ leaves a residual R' and opens X bins from the LP, where*

$$\mathrm{SIZE}(R') \le \tfrac{1}{2}\,\mathrm{SIZE}(R), \qquad X \le \mathrm{LIN}(R) - \mathrm{LIN}(R') + 1.$$

Proof. Lemma 4.1 gives $m(J) \le \mathrm{SIZE}(R)/2$ and $J \preceq R$, so $\mathrm{LIN}(J) \le \mathrm{LIN}(R)$. Substitute these bounds in Lemma 4.2. $\square$

The complete pseudocode for Karmarkar–Karp Algorithm 2 is given in Appendix H.

Worked round. Consider an input whose first two groups are

$$G_1 = \{0.8, 0.7, 0.65\}, \qquad G_2 = \{0.49, 0.46, 0.43, 0.41, 0.4\}.$$

Let $G_3, \dots, G_6$ each contain five items of size 0.4 and let $H = \{0.2\}$. Here $\mathrm{SIZE}(I) = 12.54 > 1{+}2L = 11$ and all items exceed g. Trimming G_2 to the length of G_1 and rounding gives $J = \{0.49 \times 3, 0.4 \times 20\}$. Pack $E = G_1 \uplus \{0.41, 0.4, 0.2\}$ separately.

A bin holds at most two rounded items, so $\mathrm{LIN}(J) \ge 23/2$. The solution x below attains this bound. The matrix shows the two layouts it uses, with rows for sizes 0.49 and 0.40.

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}, \quad b = (3, 20)^\top, \quad x = (1.5, 10)^\top,$$
$$\lfloor x \rfloor = (1, 10)^\top, \qquad A\lfloor x \rfloor = (2, 20)^\top,$$
$$[b - A\lfloor x \rfloor]_+ = (1, 0)^\top = A(x - \lfloor x \rfloor).$$

The 11 whole bins leave demand for one 0.49 slot. Half a copy of the two-slot layout covers it. If we fill the whole bin's 0.49 slots with originals 0.49 and 0.46, the remaining 0.43 item takes one final Next-Fit bin.

4.3 Approximation guarantee and running time

Theorem 4.4. *Algorithm 2 runs in polynomial time and returns a packing with*

$$\mathrm{ALG}(I) \le \mathrm{OPT}(I) + O\big(\log^2(\mathrm{OPT}(I) + 1)\big).$$

Proof. Originals fit their rounded slots. The separate packing of E, final Next-Fit pass, and reinsertion of S cover the other items. By the Next-Fit bound in Equation (1), proved in Section 2.2.2, $\mathrm{NF}(U) \le 2\,\mathrm{SIZE}(U) + 1$. Thus $\mathrm{SIZE}(I) \le 2$ costs a constant number of bins, with zero for empty input. Assume $\mathrm{SIZE}(I) > 2$.

Number of rounds. Let R_i be the residual after i rounds, with R_0 the items larger than g. Corollary 4.3 gives $\mathrm{SIZE}(R_i) \le 2^{-i}\,\mathrm{SIZE}(I)$, so at most $\lceil \log_2 \mathrm{SIZE}(I) \rceil$ rounds occur. After the final round t, the stopping condition gives $\mathrm{SIZE}(R_t) \le 1 + 2L$, and Next-Fit uses at most $3 + 4L$ bins for it.

Bin count before reinsertion. Let X_i be the LP bin count in the round starting from R_i, and J_i, E_i its grouping outputs. By Corollary 4.3,

$$\begin{aligned}\sum_{i=0}^{t-1} X_i &\le \sum_{i=0}^{t-1} (\mathrm{LIN}(R_i) - \mathrm{LIN}(R_{i+1}) + 1) \\ &= \mathrm{LIN}(R_0) - \mathrm{LIN}(R_t) + t \le \mathrm{OPT}(I) + t.\end{aligned}$$

The fractional cost left after one round is the starting cost of the next, so these intermediate terms cancel in the sum. The last inequality uses $R_0 \preceq I$ and $\mathrm{LIN}(R_t) \ge 0$. For the separate packing, Lemma 4.1 with $k = 2$ and $\ln(2/g) = \ln(2\,\mathrm{SIZE}(I)) \le L$ gives $\mathrm{SIZE}(E_i) \le 5 + 3L$. Next-Fit therefore uses at most $2(5 + 3L) + 1 = 11 + 6L$ bins for E_i. Including the final residual, the total before reinsertion is at most $\mathrm{OPT}(I) + t(12 + 6L) + 3 + 4L = \mathrm{OPT}(I) + O(L^2)$.

Reinserting small items. If reinsertion opens a new bin, consider the item that opened the last new bin. Its size is at most g, so each earlier bin had load greater than $1 - g$ when that item arrived. With B final bins, we have $\mathrm{SIZE}(I) \ge (B - 1)(1 - g)$, hence

$$B \le \frac{\mathrm{SIZE}(I)}{1 - g} + 1 = \mathrm{SIZE}(I) + \frac{\mathrm{SIZE}(I)}{\mathrm{SIZE}(I) - 1} + 1 \le \mathrm{OPT}(I) + 3.$$

If no bin is opened, the preceding $O(L^2)$ overhead applies. Finally, $\mathrm{SIZE}(I) \le \mathrm{OPT}(I)$ gives $L = O(\log(\mathrm{OPT}(I) + 1))$.

Running time. The paper's solver uses approximate knapsack to search for violated dual constraints [3, Section 6]. Its polynomial time bound $T(m, n)$ for m types and n items applies with tolerance one and sizes at least $1/n$ since $g = 1/\mathrm{SIZE}(I) \ge 1/n$. Scanning items and positive configurations takes $O(n + m(J_i)^2)$ per round outside the solve. Over $O(\log n)$ rounds, $m(J_i) \le n/2^{i+1}$, so the positive powers of m in T sum geometrically and absorb $m(J_i)^2$, with logs bounded at the first round. Sorting and reinsertion with a balanced tree of free capacities give $O(T(\lceil n/2 \rceil, n) + n \log n)$ total time. □

5 Conclusion

We studied the de la Vega–Lueker and Karmarkar–Karp algorithms, explaining their grouping and rounding techniques and deriving their approximation guarantees and running-time bounds. The de la Vega–Lueker algorithm gives an APTAS using at most $(1 + \varepsilon)\,\mathrm{OPT}(I) + O(\varepsilon^{-2})$ bins for any fixed $\varepsilon > 0$. Karmarkar–Karp achieves the stronger additive guarantee $\mathrm{OPT}(I) + O(\log^2(\mathrm{OPT}(I) + 1))$ in polynomial time.

By studying these two algorithms, we learned how grouping reduces the number of distinct item sizes, allowing a polynomial-size configuration ILP for fixed ε in the de la Vega–Lueker scheme. LP relaxation and careful rounding then produce a feasible packing with a provable approximation guarantee.

Beyond our scope, Hoberg and Rothvoss [2] improved the additive guarantee to $O(\log \mathrm{OPT}(I))$ using discrepancy theory and a two-stage construction that packs items into containers and then containers into bins. This remains the best known polynomial-time additive approximation guarantee. Understanding these discrepancy methods would be a natural extension of our study of LP rounding.

Recent work also establishes limits on further improvements. OpenAI's September 2026 preprint [4] constructs instances with arbitrarily large gaps $\mathrm{OPT}(I) - \mathrm{LIN}(I)$, disproving the Modified Integer Round-Up Conjecture $\mathrm{OPT}(I) \le \lceil \mathrm{LIN}(I) \rceil + 1$. It also proves that no deterministic polynomial-time algorithm can guarantee $\mathrm{OPT}(I) + C$ bins for any fixed constant C, unless P = NP; both results hold even when every item exceeds $1/6$. These results show that a constant additive loss is unattainable in general under P ≠ NP, while leaving room to improve the logarithmic upper bound. Their proofs are beyond the scope of this report.

References

[1] W. Fernandez de la Vega and G. S. Lueker. Bin packing can be solved within $1 + \varepsilon$ in linear time. *Combinatorica*, 1981. doi: 10.1007/bf02579456.

[2] Rebecca Hoberg and Thomas Rothvoss. A logarithmic additive integrality gap for bin packing. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2616–2625. SIAM, 2017. doi: 10.1137/1.9781611974782.172.

[3] Narendra Karmarkar and Richard M. Karp. An efficient approximation scheme for the one-dimensional bin-packing problem. In *23rd Annual Symposium on Foundations of Computer Science (Sfcs 1982)*, pages 312–320, Chicago, IL, USA, November 1982. IEEE. doi: 10.1109/SFCS.1982.61.

[4] OpenAI. Additive hardness and unbounded configuration gaps in bin packing. OpenAI mathematics preprints, September 2026. Preprint, September 24, 2026.

[5] Jiří Sgall. Online Bin Packing: Old Algorithms and New Results. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Alfred Kobsa, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Demetri Terzopoulos, Doug Tygar, Gerhard Weikum, Arnold Beckmann, Erzsébet Csuhaj-Varjú, and Klaus Meer, editors, *Language, Life, Limits*, volume 8493, pages 362–372. Springer International Publishing, Cham, 2014. ISBN 978-3-319-08018-5 978-3-319-08019-2. doi: 10.1007/978-3-319-08019-2_38.

A Bounding Bin Configurations Using Stars and Bars

Lemma A.1. *For any Bin Packing instance with m distinct item sizes, each at least $\varepsilon > 0$, the number of non-empty feasible bin configurations q satisfies*

$$q \leq \binom{k+m}{m} - 1, \qquad k = \lfloor 1/\varepsilon \rfloor.$$

Proof. Let c_j denote the number of items of size y_j in a configuration. Since each bin has unit capacity and every item has size at least ε, it contains at most k items. Thus,

$$\sum_{j=1}^{m} c_j \leq k, \qquad c_j \in \mathbb{Z}_{\geq 0}.$$

Introducing a non-negative integer slack variable c_R gives the equation

$$c_1 + c_2 + \cdots + c_m + c_R = k,$$

where all variables are non-negative integers.

This can be interpreted as distributing k identical balls among $m+1$ distinguishable bins. The first m bins represent the distinct item sizes, while the last bin represents unused item slots.

Equivalently, we arrange k identical balls and m dividers in a row. Each arrangement uniquely determines a non-negative integer solution to the equation.

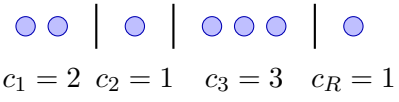


Figure 1: Stars-and-bars representation with $k = 7$ balls and $m = 3$ dividers.

As illustrated in Figure 1, each arrangement corresponds to a possible configuration of item counts.

There are $k+m$ positions, from which we choose m positions for the dividers. Hence, the number of arrangements is

$$\binom{k+m}{m}.$$

Exactly one arrangement corresponds to the empty configuration, with $c_1 = \cdots = c_m = 0$ and $c_R = k$. Excluding it leaves $\binom{k+m}{m} - 1$ possible non-empty item-count vectors. A feasible configuration must also satisfy the bin-capacity constraint

$$\sum_{j=1}^{m} c_j y_j \leq 1.$$

Some of these item-count vectors may violate the bin-capacity constraint. Every feasible configuration is nevertheless counted, so $q \leq \binom{k+m}{m} - 1$. □

B Removing Excess Items by Greedy Deflation

We describe a greedy deflation procedure for converting an integer solution satisfying the covering constraints into a packing containing exactly the required items.

Let $\mathbf{b} = \hat{\mathbf{a}}$ denote the configuration multiplicities after rounding. For each configuration $C_i = (c_{i1}, \ldots, c_{im})$, define $C_i^{(j \leftarrow x)}$ as the configuration obtained by replacing only its j-th coordinate with x.

Algorithm 1: Greedy Deflation

Input: Feasible configurations $C_1, \ldots, C_q$, required item counts $n_1, \ldots, n_m$, and integer multiplicities $\hat{\mathbf{a}}$ satisfying the covering constraints

Output: Configuration multiplicities $\mathbf{b}$ packing exactly the required items, with no increase in the number of bins

$\mathbf{b} \leftarrow \hat{\mathbf{a}}$;
for $j = 1, \ldots, m$ **do**
  $e_j \leftarrow \sum_{i=1}^{q} b_i c_{ij} - n_j$;
  while $e_j > 0$ **do**
    Choose a configuration C_i with $b_i > 0$ and $c_{ij} > 0$;
    $t \leftarrow c_{ij}$, $d \leftarrow \min\{e_j, b_i t\}$;
    $k \leftarrow \lfloor d/t \rfloor$, $r \leftarrow d \bmod t$;
    Replace k bins of configuration C_i with $C_i^{(j \leftarrow 0)}$;
    if $r > 0$ **then**
      Replace one additional bin of configuration C_i with $C_i^{(j \leftarrow t-r)}$;
    Update the corresponding multiplicities in $\mathbf{b}$;
    $e_j \leftarrow e_j - d$;
    Discard any empty bins;
return $\mathbf{b}$;

The algorithm processes excess items in batches rather than examining individual bins.

Illustrative Example. Suppose the item sizes are $y_1 = 0.1$ and $y_2 = 0.2$. The rounded solution contains 5 bins of configuration $(3, 2)$, whereas the instance requires $n_1 = 8$ and $n_2 = 7$ items.

Initially, the selected configurations contain

$$5(3, 2) = (15, 10).$$

For the first item type, the excess is $e_1 = 15 - 8 = 7$. We obtain

$$d = 7, \qquad k = 2, \qquad r = 1.$$

Hence, two bins are deflated to $(0, 2)$ and one bin to $(2, 2)$:

$$5(3, 2) \longrightarrow 2(3, 2) + 2(0, 2) + (2, 2).$$

The resulting configurations contain exactly eight items of the first type, but ten items of the second type.

For the second item type, $e_2 = 3$. Selecting the two bins of configuration $(0, 2)$ gives

$$d = 3, \qquad k = 1, \qquad r = 1.$$

One bin becomes empty and is discarded, while the other is deflated to $(0, 1)$:

$$2(3, 2) + 2(0, 2) + (2, 2) \longrightarrow 2(3, 2) + (2, 2) + (0, 1).$$

The resulting packing contains exactly

$$2(3, 2) + (2, 2) + (0, 1) = (8, 7),$$

using only four bins.

Correctness. For each item type j, the covering constraints guarantee that at least n_j items are available. Thus, the algorithm can remove exactly e_j excess items.

By definition, the deflation transformation $C_i^{(j \leftarrow x)}$ modifies only the j-th coordinate of configuration C_i, leaving all other coordinates unchanged. Consequently, for every item type $\ell \neq j$, the total number of type-ℓ items remains unchanged. In particular, the equality constraints established for previously processed item types are preserved.

Furthermore, since deflation only removes items from existing bins, it preserves bin feasibility without increasing the number of bins.

Running Time. For each of the m item types, the algorithm processes at most q configurations. Each configuration is processed using a constant number of arithmetic operations, assuming configuration transitions can be located in constant time.

Therefore, deflation requires

$$O(mq)$$

arithmetic operations, excluding the preprocessing required to index configurations.

C Monotonicity of Optimal Packing under the Instance Partial Order

Lemma C.1 (Monotonicity of Optimal Packing)**.** *Let $I = \{a_1, \ldots, a_n\}$ and $J = \{b_1, \ldots, b_m\}$ be two bin packing instances. If $I \preceq J$, then* $\mathrm{OPT}(I) \leq \mathrm{OPT}(J)$.

Proof. By the definition of $I \preceq J$, there exists an injective mapping $f : \{1, \ldots, n\} \to \{1, \ldots, m\}$ such that $a_i \leq b_{f(i)}$ for every $i \in \{1, \ldots, n\}$.

Consider an optimal packing of J using $\mathrm{OPT}(J)$ bins. For each item $a_i \in I$, replace the corresponding item $b_{f(i)}$ with a_i, and remove all unmatched items from J.

Since $a_i \leq b_{f(i)}$, the total size of items in each bin cannot increase. Hence, the resulting packing of I is feasible and uses at most $\mathrm{OPT}(J)$ bins.

Therefore,

$$\mathrm{OPT}(I) \leq \mathrm{OPT}(J).$$

□

D Bounding the Number of Nonzero Variables in an Optimal LP Solution

Proof of Lemma 2.1. Let A be the configuration matrix with m rows. The feasible region is

$$P = \{\mathbf{a} \geq \mathbf{0} : A\mathbf{a} \geq \mathbf{n}\}.$$

Since P contains no lines and the LP has a finite optimum, an optimal vertex solution $\mathbf{a}^*$ exists.

Suppose its support $S = \{j : a_j^* > 0\}$ has more than m elements. The corresponding columns of A are linearly dependent, so there is a nonzero vector $\mathbf{d}$ satisfying

$$A\mathbf{d} = \mathbf{0}, \qquad d_j = 0 \quad \text{for } j \notin S.$$

Since $a_j^* > 0$ on S, a sufficiently small $\varepsilon > 0$ ensures $\mathbf{a}^* \pm \varepsilon\mathbf{d} \geq \mathbf{0}$. Moreover,

$$A(\mathbf{a}^* \pm \varepsilon\mathbf{d}) = A\mathbf{a}^* \geq \mathbf{n}.$$

Thus, $\mathbf{a}^*$ is the midpoint of two distinct feasible solutions:

$$\mathbf{a}^* = \frac{1}{2}(\mathbf{a}^* + \varepsilon\mathbf{d}) + \frac{1}{2}(\mathbf{a}^* - \varepsilon\mathbf{d}).$$

This contradicts its being a vertex. Hence, $\mathbf{a}^*$ has at most m nonzero components. □

E Next-Fit Pseudocode

Algorithm 2: Next-Fit

Input: Items $a_1, \ldots, a_n$ of I, in the given order
Output: A packing $(B_1, \ldots, B_b)$ and its bin count $b = \mathrm{NF}(I)$
$b \leftarrow 0$, $r \leftarrow 0$;
// r is the remaining capacity of the current bin
for $i = 1, \ldots, n$ **do**
    if $a_i > r$ **then**
        $b \leftarrow b + 1$;
        Open an empty bin B_b and set $r \leftarrow 1$;
    Place item i in B_b;
    $r \leftarrow r - a_i$;
return $\big((B_1, \ldots, B_b), b\big)$;

F First-Fit Pseudocode

Algorithm 3: First-Fit

Input: Items $a_1, \ldots, a_n$ of I, in the given order
Output: A packing $(B_1, \ldots, B_b)$ and its bin count $b = \mathrm{FF}(I)$
$b \leftarrow 0$;
for $i = 1, \ldots, n$ **do**
    $j \leftarrow b + 1$;
    for $k = 1, \ldots, b$ **do**
        if $a_i \leq r_k$ **then**
            $j \leftarrow k$;
            break;
    if $j = b + 1$ **then**
        $b \leftarrow b + 1$;
        Open an empty bin B_b and set $r_b \leftarrow 1$;
    Place item i in B_j and set $r_j \leftarrow r_j - a_i$;
return $\big((B_1, \ldots, B_b), b\big)$;

G De La Vega–Lueker Pseudocode

Algorithm 4: De La Vega–Lueker

Input: Items $a_1, \ldots, a_n$ of I, and a parameter $\varepsilon > 0$
Output: A packing $(B_1, \ldots, B_b)$ of I and its bin count b
$\delta \leftarrow \varepsilon/(\varepsilon + 2)$, $t \leftarrow \lceil \delta^{-2} \rceil$;
$S \leftarrow \{a_i : a_i < \delta\}$, $L \leftarrow I \setminus S$, $h \leftarrow \lfloor |L|/t \rfloor$;
if $h = 0$ **then**
  $R \leftarrow L$, $J \leftarrow \varnothing$;
else
  Select the group boundaries of L in non-increasing order, giving $R, G_1, \ldots, G_t$, and set $J \leftarrow \varnothing$;
  for $j = 2, \ldots, t$ **do**
    Round every item of G_j up to $\min G_{j-1}$ and add it to J, recording its original;
  Solve the LP relaxation on J for an optimal basic feasible solution $\mathbf{a}^*$;
  Open $\lceil a_i^* \rceil$ bins of each configuration C_i, then deflate (Appendix B);
  Fill each slot with the original of its rounded item;
  Open one bin for each item of G_1;
Open one bin for each item of R, and let $B_1, \ldots, B_b$ be the bins opened so far;
`// Reinsertion: the pointer` p `never moves backward`
$p \leftarrow 1$;
foreach *item* $a \in S$ **do**
  while $p \le b$ ***and*** *a does not fit in B_p* **do**
    $p \leftarrow p + 1$;
  if $p > b$ **then**
    $b \leftarrow b + 1$, and open an empty bin B_b;
  Place a in B_p;
return $((B_1, \ldots, B_b), b)$;

H Karmarkar–Karp Pseudocode

Pseudocode for Karmarkar–Karp Algorithm 2

Input Instance I of items in $(0, 1]$.
Output A packing P of I.
$W \leftarrow \text{SIZE}(I)$. **If** $W \le 2$, **return** the Next-Fit packing of I.
$g \leftarrow 1/W$, $L \leftarrow \lceil \log_2(2W) \rceil$, $P \leftarrow \varnothing$.
Put all items of size at most g in S and sort the other items into R in decreasing order.
While $\text{SIZE}(R) > 1 + 2L$ **do**
  Scan R into shortest consecutive groups $G_1, \ldots, G_q$ of total size at least 2
  and an incomplete suffix H. Set $E \leftarrow G_1 \uplus H$ and $J \leftarrow \varnothing$.
  For $i = 2, \ldots, q$ **do**
    Put the smallest $|G_i| - |G_{i-1}|$ items of G_i into E.
    Add the other items of G_i to J, rounding to $\max G_i$ and recording originals.
  Append a Next-Fit packing of E to P.
  Compute a basic LP solution x on J with cost at most $\text{LIN}(J) + 1$.
  For each configuration j with $x_j \ge 1$ **do**
    Append $\lfloor x_j \rfloor$ bins with configuration j to P.
    Fill slots by rounded type with available items, placing their originals in the bins.
  Set R to the unpacked originals from J, in decreasing order.
Append a Next-Fit packing of R to P.
For each item in S, use a bin with room, opening a new bin only when needed.
Return P.

I Illustration of Linear Grouping

Figure 2 illustrates Lemma 3.1 with $\delta = 1/2$, $t = 4$, and $N = 9$, so $h = 2$ and $r = 1$. After setting aside the largest item as R, the remaining items form four consecutive groups of two items each.

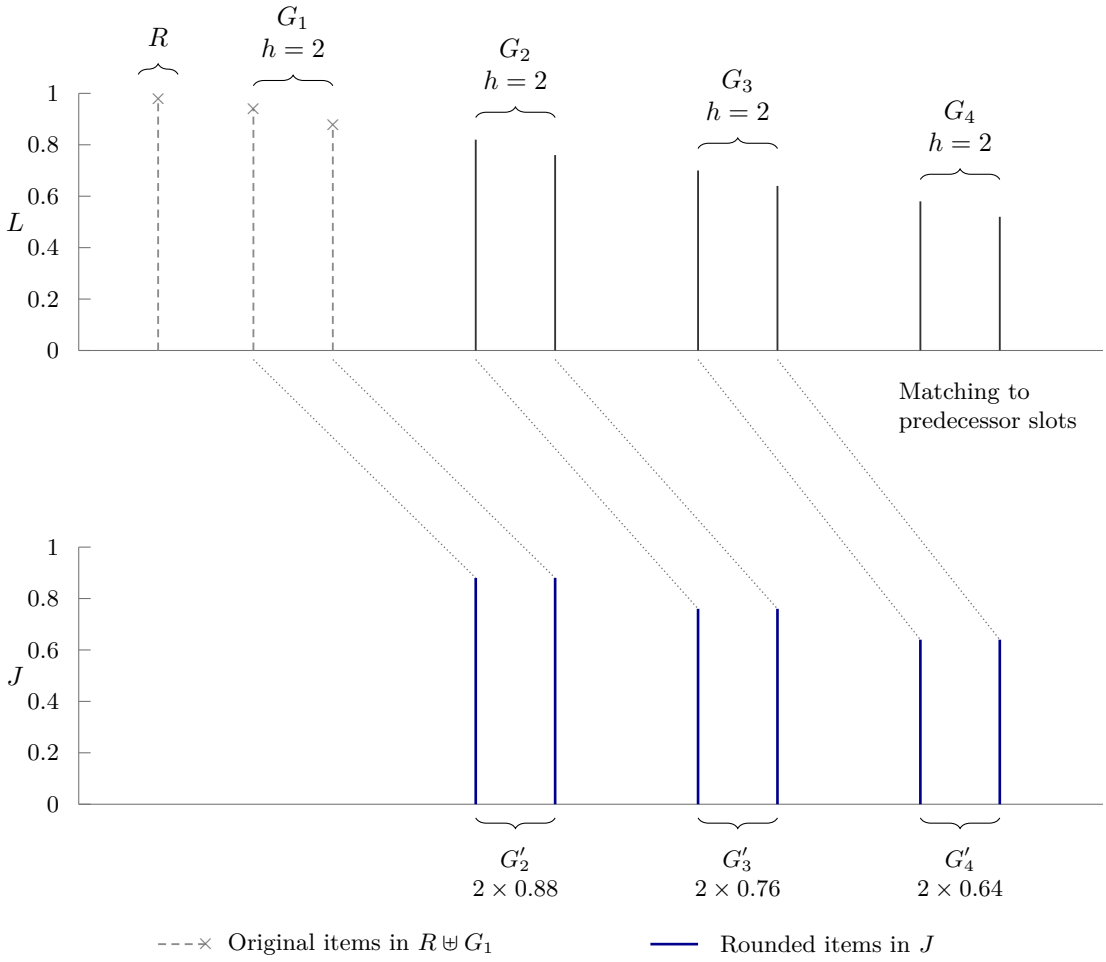


Figure 2: Before and after linear grouping. Each vertical line represents one item on a common size scale. For $j = 2, 3, 4$, round every item of G_j up to $\min G_{j-1}$ to form G'_j, keeping each copy directly beneath its original. Dotted lines match the items of G'_j to distinct slots in G_{j-1}; they illustrate $J \preceq L$, rather than the rounding operation.

Here $J = G'_2 \uplus G'_3 \uplus G'_4 = \{0.88 \times 2,\ 0.76 \times 2,\ 0.64 \times 2\}$. For example, the items 0.82 and 0.76 in G_2 round up to two copies of 0.88 in G'_2. These copies can be matched to the distinct items 0.94 and 0.88 in G_1. The three predecessor groups are disjoint, so the combined matching is injective. The items in $R \uplus G_1$ are packed separately, one per bin.

J Illustration of Geometric Grouping

Figure 3 illustrates Lemma 4.1 with $k = 2$ and $g = 0.1$. The complete groups have total sizes 2.85, 2.10, 2.00, and 2.10, respectively; each first reaches the target k at its last item. The remaining suffix H has total size $0.35 < k$.

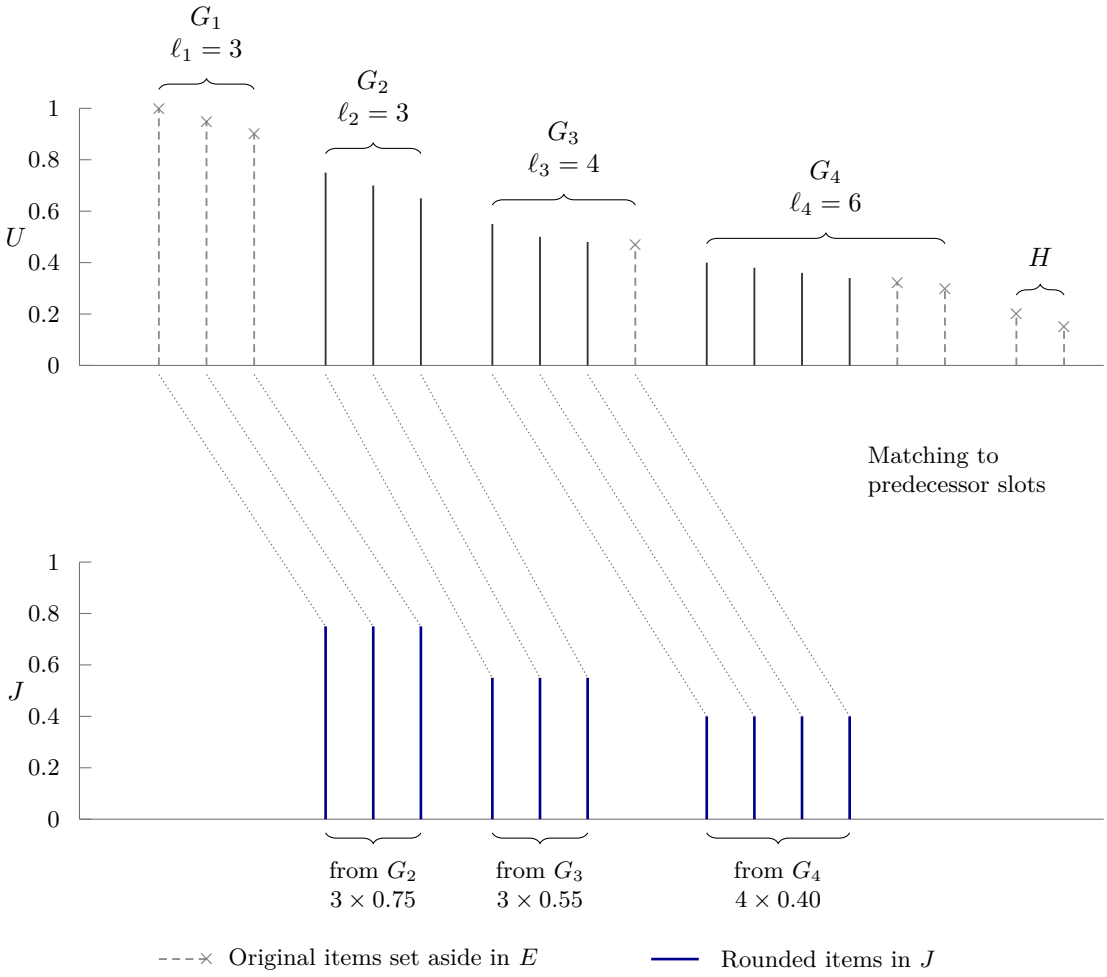


Figure 3: Before and after geometric grouping. Each vertical line represents one item, with height equal to its size on the common scale. For $i \geq 2$, retain ℓ_{i-1} items of G_i and round them up to $\max G_i$, keeping them aligned beneath their originals. Dotted connections match these rounded items to distinct slots in G_{i-1}, illustrating $J \preceq U$.

Here $d_2 = 0$, $d_3 = 1$, and $d_4 = 2$. Thus $D_3 = \{0.47\}$ and $D_4 = \{0.32, 0.30\}$ are trimmed, giving

$$E = G_1 \uplus D_3 \uplus D_4 \uplus H, \qquad J = \{0.75 \times 3,\ 0.55 \times 3,\ 0.40 \times 4\}.$$

The first group supplies matching slots but contributes no rounded items to J; its original items are packed separately in E.

AI Disclosure

We used Codex and Claude to help understand the papers and proofs, including obtaining algorithm overviews and explanations of intermediate steps. We also used them to review the coherence and logical flow of our exposition, assist with drafting and polishing text, and edit LaTeX. Codex assisted with drafting the conclusion and a limited search for recent results cited there. We read the original papers, worked through the essential proofs, and checked examples. We reviewed AI-assisted content line by line, revised it to reflect our understanding, and proofread the report over several rounds. We take full responsibility for the correctness and clarity of the submitted work.